PROJETO FINAL · GUIA DE CONFIGURAÇÃO

Do zero ao sistema completo

API em C#, frontend em Vue 3 e banco MySQL online no alwaysdata

Vue 3 · o que o usuário vê

API C# · as regras do sistema

MySQL no alwaysdata · os dados

A ARQUITETURA

Três partes, cada uma com uma função



O frontend **só conversa com a API**, por HTTP e JSON.

Só a API **conhece a senha** e fala com o banco.

O banco não roda no seu PC: ele fica **na internet**.

ANALOGIA

Pense num restaurante

FRONTEND

O salão

Cardápio, mesas e decoração. É onde o cliente faz o pedido e recebe o prato.

API

A cozinha

Recebe o pedido, segue as receitas (regras) e prepara a resposta.

BANCO

A despensa

Guarda os ingredientes. Só a cozinha tem a chave.

O cliente nunca entra na despensa. Do mesmo jeito, **o navegador nunca fala direto com o banco**.

ANTES DE COMEÇAR

O que instalar no computador

FERRAMENTA	PARA QUE SERVE	TESTE NO TERMINAL
Docker Desktop	Compila e roda a API e o site	`docker --version`
Git	Salvar versões e entregar	`git --version`
VS Code	Editor de código	Vue (Official), REST Client, Docker
Conta alwaysdata	O banco MySQL online, grátis	admin.alwaysdata.com

Não precisa instalar .NET nem Node.js. As imagens do Docker trazem o compilador do C# e o Node, e baixam as bibliotecas durante o build.

ORGANIZAÇÃO

Uma pasta, dois containers

```
projeto-final/
├── backend/                  # API C# → container api
│   ├── Models/Produto.cs
│   ├── Program.cs
│   ├── Api.csproj · appsettings.json
│   ├── Api.http
│   └── Dockerfile
├── frontend/                 # Vue 3 → container web
│   ├── src/ (services, views, router)
│   ├── package.json · vite.config.js
│   ├── index.html · nginx.conf
│   └── Dockerfile
├── banco/schema.sql
├── .env                      # senha, fora do Git
├── compose.yaml
├── CLAUDE.md
└── README.md
```

backend/

Projeto .NET, compilado pelo Docker.

frontend/

Projeto Vue, gerado pelo Docker e servido pelo nginx.

Raiz

O que une as partes: banco, senha, compose.yaml e documentação.

O ROTEIRO

Oito etapas, sempre testando no fim de cada uma

01
Banco
Tabela no alwaysdata

02
API
Rotas CRUD em C#

03
Segredo
Connection string protegida

04
Subir a API
Docker e arquivo .http

05
Vue
Arquivos e container

06
Conectar
Vue chamando a API

07
Rotina
Comandos do Docker

08
Entrega
Git, README e checklist

ETAPA 1 · BANCO

Criar o banco e as tabelas no alwaysdata

1. No painel, **Databases › MySQL**: crie o banco **conta_projeto**.

2. Na aba **Users**, confirme o usuário e a senha com acesso ao banco.

3. Abra o **phpMyAdmin**, aba **SQL**, e rode o **banco/schema.sql**.

4. Anote host, banco, usuário e senha. Eles viram a connection string.

banco/schema.sql

```sql
CREATE TABLE produtos (
  id    INT AUTO_INCREMENT PRIMARY KEY,
  nome  VARCHAR(100)  NOT NULL,
  preco DECIMAL(10,2) NOT NULL
);

INSERT INTO produtos (nome, preco)
VALUES ('Caderno', 19.90),
       ('Caneta', 3.50);
```

Teste: na aba Visualizar do phpMyAdmin aparecem os dois produtos.

ETAPA 2 · API

Criar a API sem instalar nada

backend/Api.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Dapper" Version="2.1.66" />
    <PackageReference Include="MySqlConnector" Version="2.6.2" />
  </ItemGroup>
</Project>
```

backend/Models/Produto.cs

```
namespace Api.Models;

public class Produto
{
  public int Id { get; set; }
  public string Nome { get; set; } = "";
  public decimal Preco { get; set; }
}
```

Sem **dotnet new**: você escreve os arquivos e o Docker baixa o **Dapper** e o **MySqlConnector** no build.

ETAPA 2 · API

Program.cs: conexão e CORS

```
var builder = WebApplication.CreateBuilder(args);

// 1. Connection string (vem do .env, pelo Docker)
var conexao = builder.Configuration.GetConnectionString("Padrao")
    ?? throw new InvalidOperationException(
        "Connection string 'Padrao' não encontrada.");

// 2. CORS: autoriza o Vue a chamar a API
builder.Services.AddCors(opcoes =>
    opcoes.AddDefaultPolicy(politica => politica
        .WithOrigins("http://localhost:3000")
        .AllowAnyHeader()
        .AllowAnyMethod()));

var app = builder.Build();
app.UseCors();
```

conexao é lida da configuração. Se estiver vazia, a API para com uma mensagem clara.

CORS é a lista de sites que podem chamar a API: o nosso site Vue, em localhost:3000.

No topo do arquivo: **using Api.Models; using Dapper; using MySqlConnector;**

ETAPA 2 · API

As cinco rotas do CRUD

VERBO	ROTA	O QUE FAZ	RESPOSTA
GET	/api/produtos	Lista todos	200 + lista
GET	/api/produtos/{id}	Busca um pelo id	200, ou 404 se não existe
POST	/api/produtos	Cadastra um novo	201 + o criado
PUT	/api/produtos/{id}	Altera um existente	204, ou 404
DELETE	/api/produtos/{id}	Exclui	204, ou 404

```
app.MapGet("/api/produtos", async () =>
{
    await using var con = new MySqlConnection(conexao);
    return await con.QueryAsync<Produto>("SELECT id, nome, preco FROM produtos ORDER BY nome");
});
```

ETAPA 2 · API

Gravar no banco com segurança

```
app.MapPost("/api/produtos", async (Produto produto) =>
{
    await using var con = new MySqlConnection(conexao);
    produto.Id = await con.ExecuteScalarAsync<int>(
        "INSERT INTO produtos (nome, preco) VALUES (@Nome, @Preco);" +
        "SELECT LAST_INSERT_ID();", produto);
    return Results.Created($"/api/produtos/{produto.Id}", produto);
});
```

(Produto produto)

O ASP.NET lê o JSON enviado pelo Vue e monta o objeto.

@Nome e @Preco

Parâmetros. O Dapper preenche com segurança: nada de SQL Injection.

PUT e DELETE

Mesmo molde, com ExecuteAsync. Código completo no PDF.

ETAPA 3 · SEGREDO

A senha nunca entra no código

.env, na raiz do projeto

```
ConnectionStrings__Padrao=Server=mysql-conta.alwaysdata.net;
Database=conta_projeto;User ID=conta;Password=SUA_SENHA
```

Quebrado aqui só para caber: no arquivo, é uma linha só.

.gitignore

```
.env
bin/
obj/
node_modules/
dist/
```

O Docker entrega a senha

O compose.yaml lê o .env e passa o valor para o container da API.

Por que “__”?

Dois sublinhados viram “:”. O .NET lê ConnectionStrings:Padrao.

Modelo sem senha

O .env.example vai para o Git; o .env, nunca.

ETAPA 4 · SUBIR A API

Testar a API antes de qualquer tela

compose.yaml (por enquanto)

```
services:
  api:
    build: ./backend
    ports: ["8080:8080"]
    env_file: .env
```

Na raiz do projeto

```
docker compose up --build
```

1. Espere o build. O terminal fica mostrando os logs: é normal.

2. Abra localhost:8080: deve aparecer “API do projeto final no ar!”.

3. No backend/Api.http, clique em **Send Request** em cada chamada.

4. Confira no phpMyAdmin: a Mochila está na tabela.

Por que testar agora? Se a tela der erro depois, você já sabe que a API e o banco funcionam.

ETAPA 5 · VUE

Criar o frontend com Vue 3

frontend/package.json (resumido)

```
{
  "name": "frontend",
  "type": "module",
  "scripts": { "build": "vite build" },
  "dependencies": {
    "vue": "^3.5.0",
    "vue-router": "^5.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^6.0.0",
    "vite": "^8.0.0"
  }
}
```

Os outros arquivos de base

index.html	a única página
vite.config.js	configura o Vite
src/main.js	liga o app
src/App.vue	a moldura
src/router/	tela por URL
src/views/	as telas
src/services/	fala com a API

Sem **npm create**: o conteúdo de cada arquivo está no PDF.

ETAPA 5 · VUE

O site também vira um container

frontend/Dockerfile

```
# etapa 1: baixar pacotes e gerar o site
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

# etapa 2: servir com nginx
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
```

compose.yaml completo (na raiz)

```
services:
  api:
    build: ./backend
    ports: ["8080:8080"]
    env_file: .env
  web:
    build:
      context: ./frontend
      args:
        VITE_API_URL: http://localhost:8080
    ports: ["3000:80"]
    depends_on: [api]
```

Sem serviço de banco: ele está no alwaysdata.

ETAPA 6 · CONECTAR

services/api.js: o mensageiro

```
const API = import.meta.env.VITE_API_URL

async function pedir(caminho, opcoes = {}) {
  const resposta = await fetch(`${API}${caminho}`, {
    headers: { 'Content-Type': 'application/json' },
    ...opcoes,
  })
  if (!resposta.ok) {
    throw new Error(`A API respondeu com erro ${resposta.status}`)
  }
  return resposta.status === 204 ? null : resposta.json()
}

export const listarProdutos = () => pedir('/api/produtos')
export const criarProduto = (produto) =>
  pedir('/api/produtos', { method: 'POST', body: JSON.stringify(produto) })
```

No compose.yaml (build)

```
VITE_API_URL:
http://localhost:8080
```

O Vue só enxerga variáveis que começam com **VITE_**.

Toda chamada passa por **pedir()**: o erro é tratado num lugar só.

ETAPA 6 · CONECTAR

A tela: a lógica em <script setup>

```
import { ref, onMounted } from 'vue'
import {
  listarProdutos, criarProduto, excluirProduto,
} from '@/services/api'
const produtos = ref([])
const novo = ref({ nome: '', preco: 0 })
const erro = ref('')

async function carregar() {
  try {
    erro.value = ''
    produtos.value = await listarProdutos()
  } catch {
    erro.value = 'Erro ao carregar. A API está rodando?'
  }
}
```

```
async function salvar() {
  await criarProduto(novo.value)
  novo.value = { nome: '', preco: 0 }
  await carregar()
}

async function excluir(id) {
  await excluirProduto(id)
  await carregar()
}

// quando a tela abre
onMounted(carregar)
```

ETAPA 6 · CONECTAR

A tela: o HTML em <template>

```
<h1>Produtos</h1>
<form @submit.prevent="salvar">
  <input v-model="novo.nome" placeholder="Nome" required />
  <input v-model.number="novo.preco" type="number"
         step="0.01" min="0" required />
  <button type="submit">Adicionar</button>
</form>
<p v-if="erro">{{ erro }}</p>
<ul>
  <li v-for="p in produtos" :key="p.id">
    {{ p.nome }}: {{ moeda(p.preco) }}
    <button @click="excluir(p.id)">Excluir</button>
  </li>
</ul>
```

{{ }} mostra o valor de uma variável.

v-model liga o campo à variável: digitou, mudou.

@submit / @click chamam uma função no evento.

v-for repete o <li> para cada produto.

v-if só mostra o erro se ele existir.

Teste: docker compose up --build e abra localhost:3000. A lista aparece e o cadastro grava no alwaysdata.

O ERRO MAIS COMUM

CORS: o porteiro do navegador

```
Access to fetch at 'http://localhost:8080/api/produtos' from origin 'http://localhost:3000' has been blocked by CORS policy
```

1 · O navegador

A página veio de **localhost:3000** e quer chamar **localhost:8080**. Porta diferente = outra origem.

→

2 · A pergunta

Antes de entregar a resposta, o navegador confere: a API autorizou essa origem?

→

3 · A solução

Na API, **WithOrigins(...)** autoriza localhost:3000. Sem isso, bloqueado.

O .http e o navegador direto na API funcionam, só o Vue falha? **É CORS.** Confira as origens no Program.cs.

ETAPA 7 · ROTINA

O dia a dia com Docker

COMANDO, NA RAIZ	QUANDO USAR
`docker compose up --build`	Depois de qualquer mudança: reconstrói e liga tudo.
`docker compose up --build web`	Mudou só o Vue: reconstrói só o site.
`docker compose up --build api`	Mudou só o C#: reconstrói só a API.
`docker compose logs -f api`	Ver erros do C# e do banco.
`docker compose down`	Desliga tudo.

Por que **VITE_API_URL** é **localhost:8080** e não **api:8080**? Porque quem chama a API é o **navegador**, que roda no seu PC, fora do Docker.

ETAPA 8 · ENTREGA

Checklist antes de entregar

- **.env fora do Git.** Confira com git status: ele não pode aparecer.
- **banco/schema.sql** recria todas as tabelas.
- **Todas as rotas** testadas no arquivo .http.
- **Sem senha no código** nem em prints da apresentação.
- **.env.example presente,** com valores de exemplo.
- **README.md** explica como rodar, passo a passo.
- **docker compose up --build** funciona numa pasta recém-clonada.
- **Você sabe explicar** cada arquivo para a banca.

SE ALGO DER ERRADO

Erros comuns: onde olhar primeiro

SINTOMA	CAUSA PROVÁVEL E SOLUÇÃO
“blocked by CORS policy” no console	Origem do Vue fora do WithOrigins, ou UseCors() faltando.
API não sobe: connection string não encontrada	O .env está faltando, fora da raiz ou com a chave errada.
Access denied for user	Usuário ou senha errados, ou sem permissão no banco.
Tela mostra “Erro ao carregar”	API parada ou VITE_API_URL errado. Veja docker compose ps.
Mudança no código não aparece	Faltou o --build: rode docker compose up --build.
“port is already allocated”	Outro programa usa a porta 3000 ou 8080. Pare-o ou troque a porta.

RESUMO

O Vue pede, a API decide, o banco guarda.

Uma etapa por vez

Banco, API, teste, Vue, rotina. Teste no fim de cada uma.

Senha só na API

Só no .env, entregue pelo Docker. Nunca no código, nunca no Git.

Próximo passo

Como pedir cada etapa ao Claude Code, e entender o que ele fez.